Abstract

Contents

1 Introduction

1.1 Context and Motivation

1.2 Research Objectives

1.3 Main Contributions of the Thesis

1.4 Structure of the Document

2 Theoretical Background

The field of artificial intelligence has undergone deep transformations since its inception,evolving through distinct paradigms that reflect different philosophical and technical approaches to machine intelligence. This chapter provides the theoretical foundations necessary to understand the framework proposed in this thesis, exploring the evolution of learning paradigms in AI, some techniques of visual perception and the principles underlying symbolic reasoning and reinforcement learning.

2.1 The Evolution of Learning in Artificial Intelligence

2.1.1 Artificial Intelligence Paradigms

The history of artificial intelligence is characterized by the emergence and evolution of three major paradigms, each representing a distinct approach to providing machines the human-like reasoning abilities. The symbolic paradigm, dominant from the 1950s through the 1980s. This paradigm was grounded in the hypothesis that human intelligence could be replicated through the manipulation of symbols, according to logical rules, to perform reasoning tasks. The systems built on this paradigm operated on explicit knowledge representations, using formal logic and rule-based reasoning to solve problems. These systems are grounded in formal logic and manually designed rule sets. Classic examples include expert systems such as MYCIN and Deep Blue, as well as semantic networks and logic programming frameworks [1]. They are capable of performing reasoning tasks and have achieved notable success in constrained domains where knowledge can be explicitly encoded. However, the symbolic paradigm faced significant limitations due to its high reliance on manually encoded knowledge. Indeed, when faced with tasks requiring pattern recognition, learning from experience, the management of uncertainty or incomplete information, it suffered from brittle-

ness [1, 2]. These challenges gave rise to the connectionist paradigm, which gained prominence in the late 1980s and experienced a resurgence in the 2010s with the advent of deep learning [3]. Inspired by the structure and function of biological neural networks, connectionist systems learn representations directly from data through training processes that adjust connection weights between artificial neurons [3,4]. This paradigm has achieved remarkable success in tasks such as image recognition, natural language processing and game playing, often surpassing human-level performance in specific domains [5]. Their capacity for robust and interpretable reasoning remains limited. Deep learning still struggles with structured reasoning, causal inference and factual consistency. Despite the impressive capabilities of neural networks, their operation and internal representation often lacks transparency, giving rise to what is commonly referred to as the "black box problem" [2], which limits interpretability, accountability and trust. Indeed, many of these approaches compromised interpretability and relied on strong assumptions. Moreover, they lacked the rich semantic expressiveness characteristic of symbolic systems. These limitations provide strong motivation for integrating neural and symbolic paradigms. In short, symbolic systems excel at reasoning but lack at perception, while neural networks excel at perception but struggle with reasoning [1]. This limitation has motivated the emergence of a hybrid paradigm, which seeks to combine the learning capabilities of neural networks with the interpretability and reasoning power of symbolic systems. Neuro-symbolic AI represents this third wave, aiming to bridge the gap between data-driven learning and knowledge-based reasoning as well as between logic-based inference and gradient-based learning. By integrating these paradigms, it aims to potentially offer systems that are both powerful and explainable [6]. Neuro-symbolic methods have demonstrated significant promise across a wide range of application areas, especially in scenarios that demand the combination of data-driven learning and structured symbolic reasoning [6]. This emerging paradigm reflects a synthesis of complementary approaches, bringing together the statistical strengths of deep learning, the formal foundations of symbolic logic and the adaptability of large-scale pretrained models.
Nonetheless, several important challenges remain. These include:

- Ensuring compatibility between the discrete symbolic structures used in symbolic approaches and the continuous vector representations typical of neural networks.
- Enabling the model to dynamically learn, adapt, modify, and refine logical rules.

- Achieving an optimal balance between system complexity and computational efficiency.
- Addressing the absence of unified architectural frameworks by developing a general architecture that systematically and reusably integrates neural and symbolic components.

Although several challenges persist, neuro-symbolic AI is widely regarded as a promising direction for developing systems capable of transparent, reliable and generalizable reasoning.

2.1.2 Symbolic and Hybrid Approaches in Artificial Intelligence

Symbolic AI operates on the principle that intelligence can be reduced to the manipulation of symbols representing concepts, objects and relationships. The knowledge in symbolic systems is typically encoded using various representation schemes, including production rules, semantic networks, frames and logical predicates. These representations enable explicit reasoning processes such as deduction, induction and abduction, allowing systems to draw inferences, explain their decisions and incorporate domain knowledge provided by human experts. The advantages of symbolic approaches include interpretability, the ability to incorporate prior knowledge and support for logical reasoning and explanation. However, they also present significant challenges: knowledge acquisition bottleneck, brittleness when facing situations not covered by explicit rule and difficulty in learning from raw sensory data. The manual encoding of knowledge is intensive and may not capture the full complexity of real-world domains [1]. Hybrid approaches seek to overcome these limitations by integrating symbolic and connectionist methods. Various integration strategies have been proposed, including systems where neural networks generate symbolic representations that are then processed by reasoning engines, architectures that embed symbolic knowledge into neural network structures and frameworks that use symbolic reasoning to guide neural learning processes [6]. Recent work in neuro-symbolic AI has demonstrated promising results in domains requiring both pattern recognition and logical reasoning, such as visual question answering, knowledge graph completion, and program synthesis. Following the advent of neuro-symbolic artificial intelligence, a paradigm shift has been induced in the way machines learn. This approach combines deductive, rule-based learning, typical of symbolic artificial intelligence, with the inductive and pattern-recognition capabilities characteristic of neural networks. The result is a

hybrid approach that leverages the strengths of both domains, leading to a more comprehensive learning methodology. Neuro-symbolic concepts demonstrate strong compositional generalization: new concepts can be created by structurally combining previously learned or existing ones [7]. This compositionality naturally allows for a decomposition of the learning problem, in order to learn and recognize basic concepts. During the inference phase, these concepts can be recombined to achieve the objectives of more complex tasks. Neuro-symbolic AI is one of the most exciting directions in AI research today. It aims to create intelligent systems capable of learning and reasoning like humans.

2.1.3 Reinforcement Learning: Learning Through Experience

Reinforcement learning (RL) represents a distinct learning paradigm in which an agent learns to make decisions by interacting with an environment. Unlike supervised learning, which requires labeled training data, or unsupervised learning, which seeks to discover patterns in unlabeled data, reinforcement learning is based on the principle of learning from the consequences of actions through trial and error [8]. The agent takes actions, observes their consequences, and receives feedback in the form of rewards. At the beginning of learning, an agent may have limited or no prior knowledge of the environment. In model-free approaches, no explicit model is available, and learning relies entirely on interaction. In contrast, model-based methods assume a known model or aim to learn it explicitly. This flexibility makes reinforcement learning suitable for complex environments, where providing a complete dataset or manually specifying optimal behavior is impractical. The core structure of Reinforcement Learning (RL) is modelled through a Markov Decision Process (MDP), which provides a mathematical framework for sequential decision-making [8]. An MDP is defined by a finite or continuous set of states representing the possible situations an agent can encounter, a set of actions available to the agent and a transition function that specifies the probability of moving from one state to another after taking a particular action. Additionally, a reward function assigns numerical feedback to each state-action pair, quantifying the immediate benefit of a given choice. By observing the current state, selecting an action, and receiving a reward and a new state, the agent incrementally improves its decision-making strategy [9]. Over time, the agent aims to learn an optimal policy, which is a strategy mapping states to actions. The goal of this policy is to maximize the expected cumulative reward across future interactions, balancing short-term

gains with long-term benefits through exploration and exploitation. Several agent design approaches exist, including methods based on value estimation and approaches that learn action values directly, such as Q-learning. More advanced techniques combine different strategies to improve learning efficiency. Due to its ability to learn from experience, reinforcement learning has achieved notable success in domains such as game playing, robotics, and resource management [9].

2.2 Computer Vision Techniques

Visual perception is fundamental to many AI applications, enabling systems to extract meaningful information from images and video streams. One of the most relevant goals of computer vision is to mimic human visual perception. Robustness is commonly assessed by comparing algorithmic performance to human-level perception under similar conditions. In this context, robustness refers to the ability to extract task-relevant visual information, even when this information is sparse, corrupted, or significantly different from a previously observed representation, while maintaining tolerance to outliers. This section explores both classical and modern approaches to video analysis and feature extraction.

2.2.1 Classical Techniques for Video Analysis

Before the advent of deep learning, computer vision systems primarily relied on deterministic techniques and manually engineered features. Traditional algorithms such as edge detection methods, threshold-based segmentation, and morphological processing were widely employed to extract structural information from images. Later, handcrafted feature descriptors including SIFT [10] (Scale-Invariant Feature Transform), SURF [11] (Speeded-Up Robust Features), and HOG [12] (Histograms of Oriented Gradient) were introduced to capture local patterns useful for object recognition and matching tasks. Although these methods achieved notable success, they suffered from several inherent limitations, particularly in their sensitivity to noise, changes in illumination, and poor generalization across diverse environments [10]. As a result, significant domain expertise and extensive manual effort were required to design robust feature representations, which constrained their scalability and adaptability to complex real-world scenarios.
Originally, computer vision relied heavily on handcrafted features and algorithmic techniques designed based on understanding of visual perception and signal processing. These principles underpin a classical computer vision

pipeline, typically composed of preprocessing, feature extraction, and pattern recognition stages.
Preprocessing techniques include operations such as noise reduction, contrast enhancement, and colour space transformations that prepare raw visual data for subsequent analysis. Edge detection algorithms, such as the Canny edge detector and Sobel operator, identify boundaries between regions with different intensity characteristics, providing fundamental structural information about objects in the scene.
Feature extraction methods in classical computer vision include corner detectors like Harris corners [13] and FAST [14] (Features from Accelerated Segment Test), which identify points of interest that can be reliably detected across different views. Descriptors such as SIFT and SURF provide representations of local image patches that are robust to changes in scale, rotation, and illumination. These features enable tasks such as object recognition, tracking, and image matching. Although these classical techniques have been largely superseded by deep learning methods in many applications, they remain valuable in contexts where computational resources are limited, interpretability is crucial, or training data is scarce. Moreover, the principles underlying these techniques continue to inform modern approaches and provide insights into visual perception.

2.2.2 Visual Perception in Dynamic Environments

The analysis of dynamic environments is particularly relevant for applications such as autonomous navigation, human-robot interaction, video surveillance, and sports analytics. In the context of video games, which serve as testbeds for this thesis, dynamic environments present controlled yet complex scenarios where agents must perceive, reason about, and respond to changing situations in real-time. Understanding dynamic environments requires not only processing individual images but also tracking changes over time and recognizing temporal patterns. Visual perception in such contexts involves several key challenges: detecting and segmenting objects, tracking their motion across frames, recognizing actions and behaviors, and predicting future states.
Object detection and segmentation are fundamental capabilities for understanding scenes. Object detection identifies the presence and location of objects within images, typically using bounding boxes to localize them. Semantic segmentation assigns a class label to each pixel, providing a more detailed understanding of scene structure. Instance segmentation combines both tasks, identifying individual object instances and their precise boundaries.

Temporal reasoning in video analysis requires integrating information across multiple frames. Recurrent neural networks and their variants, such as Long Short-Term Memory (LSTM) networks [15] , have been widely used to capture temporal dependencies. Three-dimensional convolutional networks extend spatial convolutions into the temporal dimension, enabling the extraction of spatiotemporal features directly from video data. In this work, the temporal complexity of video is addressed in a modular manner, by first extracting spatial features from individual frames and delegating temporal modeling to subsequent stages.

2.2.3 Deep Learning for Feature Extraction from Video Streams

The advent of Deep Learning has revolutionized the field of Computer Vision, enabling models to automatically learn hierarchical representations of images and enabling end-to-end learning of feature representations directly from raw pixel data through CNNs (Convolutional Neural Networks).

Convolutional Neural Networks for Visual Representation Learning

Convolutional Neural Networks (CNNs or ConvNets) form the foundation of modern visual recognition systems and represent a major paradigm shift in computer vision. It enabled models to automatically learn hierarchical representations, directly from raw data, where lower layers capture low-level features such as edges and textures, while higher layers represent more abstract concepts and object categories. Convolutional Neural Networks are designed to process data represented as multidimensional arrays. These structures include one-dimensional signals such as time series and language sequences, two-dimensional data such as images, and three-dimensional data such as videos. The effectiveness of CNNs is rooted in four fundamental architectural principles that exploit the compositional nature of visual data: local connectivity, shared weights, pooling operations, and hierarchical multi-layer organization. Unlike fully connected networks, each convolutional neuron is connected only to a local region of the input, significantly reducing the number of parameters and improving generalization. Weight sharing allows the same convolutional filter to detect a specific visual pattern regardless of its spatial location within the image.

A typical CNN architecture is organized as a sequence of stages, each composed of convolutional layers, nonlinear activation functions, such as the Rectified Linear Unit (ReLU), and pooling layers. In convolutional layers,

neurons are grouped into feature maps, where each map applies a specific filter across the entire input, producing a representation that highlights the presence of particular local patterns. Pooling layers are responsible for locally aggregating information, reducing the spatial dimensionality of feature maps and introducing a degree of invariance to small translations and distortions in the input. The most common operation is max pooling, which selects the maximum value within a local neighborhood. The alternation of convolution, nonlinearity, and pooling enables the network to progressively construct increasingly abstract and robust representations. In the visual domain, low-level features such as edges and orientations are combined into more complex patterns, which in turn form object parts and ultimately complete objects. This hierarchical aggregation allows the learned representations to remain stable under local variations in position, shape, or appearance. In other words, CNNs leverage convolutional operations to capture local spatial patterns, which are gradually combined across deeper layers to form increasingly abstract representations. This hierarchical learning process allows the network to identify complex visual structures without relying on manually designed features. The effectiveness of CNNs was clearly demonstrated in the ImageNet challenge [57], where deep architectures achieved unprecedented performance, significantly outperforming traditional computer vision approaches. This progress has made it possible to tackle complex tasks such as image segmentation, with high accuracy.

Evolution of Deep Architectures for Visual Representation Learning

From Feature Extraction to Dense Prediction: Semantic Segmentation Architectures

2.3 Knowledge Representation and Reasoning

2.3.1 Symbolic Representations, Semantic Networks, and Interpretable Models

2.4 Reinforcement Learning Principles

This section delves deeper into the principles and methods of reinforcement learning, with particular focus on Deep Q-Networks, which play a central role in the framework proposed in this thesis.

2.4.1 Theoretical foundations

The mathematical foundation of reinforcement learning rests on the framework of Markov Decision Processes (MDPs). An MDP formalizes sequential decision-making problems in which an agent interacts with an uncertain environment and seeks to maximize a cumulative reward over time. An MDP is defined by the tuple (S, A, P, R, γ), where:

- S is the state space, i.e., the set of all possible configurations of the environment.
- A is the action space, i.e., the set of all actions available for the agent in each state.
- $P(s' \mid s, a)$ is the transition probability function, i.e., the probability that, after performing action a in state s, the system transitions to state s'.
- $R(s, a)$ is the reward function, which provides a numerical value quantifying the desirability of transitioning from state s to state s' after performing action a, assigning a scalar value to each state-action pair.
- $\gamma \in [0, 1)$ is the discount factor that determines the relative importance of immediate versus future rewards. Smaller values of γ encourage myopic behavior focused on immediate gains, whereas values closer to 1 promote long-term planning.

A fundamental assumption in MDPs is the Markov property, which states that the future evolution of the system depends solely on the current state and action, independent of the full history of past states. Mathematically:

$$P(s' \mid s, a, s_{t-1}, a_{t-1}, \dots) = P(s' \mid s, a)$$

This memoryless characteristic significantly simplifies the analysis and allows the use of dynamic programming techniques. The agent tries to maximize the total cumulative reward. This cumulative reward is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k})$$

It is not simply the sum of instant rewards, but the discounted reward over time. Due to the γ it is possible to give more weight to immediate or future rewards.

Policie

In reinforcement learning, a policy π defines the agent's behavior by specifying which action should be taken in each possible state [9]. Formally, a policy can be represented either as a function or as a probability distribution over actions. Two main types of policies can be distinguished. A deterministic policy $\pi(s)$ assigns a single, fixed action to each state. In contrast, a stochastic policy $\pi(a \mid s)$ associates a probability with each available action. The agents progressively learn an optimal policy through continuous interaction with the environment. This learning process requires collecting experience by visiting new states and testing different actions. Through this interaction, the agent refines its strategy in order to maximize the cumulative reward obtained over time. A central challenge in this process is the exploration-exploitation trade-off. Exploration encourages the agent to try new and unfamiliar actions in order to discover potentially better rewards and improve its knowledge of the environment. On the other hand, exploitation focuses on selecting actions that are already known to yield high returns, thus capitalizing on past experience. Achieving an effective balance between these two objectives is crucial for learning a robust and optimal policy.

Value Function

Value functions are used to evaluate policies and play a fundamental role in many reinforcement learning algorithms. The state-value function $V_\pi(s)$ estimates the expected cumulative reward obtained when starting from state s and following policy π.

$$V_\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})\right]$$

The action-value function $Q_\pi(s, a)$ measures the expected return after taking action a in state s and subsequently following policy π, are also called Q-values [16].

$$Q_\pi(s, a) = \mathbb{E}\left[R(s, a, s') + \gamma V_\pi(s')\right]$$

The central objective of reinforcement learning is to identify an optimal policy π^* that maximizes these value functions across all states.

Potential and areas of application

MDPs provide the theoretical foundation for a wide range of reinforcement learning algorithms, including value iteration, policy iteration, temporal-difference learning and Q-learning. These methods exploit the mathematical

structure of MDPs to iteratively improve policies, either when the environment model is known or when it must be learned from experience. As a result, MDPs play a crucial role in enabling autonomous agents to learn effective behaviors through interaction with complex and uncertain environments. Due to their strong theoretical foundation and flexibility, MDPs are extremely important across numerous application domains. In robotics and automation, for example, robots rely on MDPs to navigate uncertain and dynamic environments. Beyond robotics, MDPs are also widely applied in healthcare for treatment planning and optimization, in finance for trading strategies and risk management, and in game AI for developing intelligent and adaptive agents. This broad applicability highlights the practical significance of MDPs in real-world decision-making problems [9].

The path to success

The primary objective of reinforcement learning is to identify an optimal policy that maximizes long-term rewards. This objective leads to one of the most fundamental results in RL theory: the Bellman Optimality Equation [17]. This equation provides a recursive definition of optimal value functions, both for states and for state-action pairs [18].

- **Optimal State Value Function** $V^*(s)$:

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^*(s') \right] \tag{2.1}$$

- **Optimal Action Value Function** $Q^*(s, a)$:

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right] \tag{2.2}$$

The Bellman optimality equation removes any dependence on a particular policy. Instead, it establishes a self-consistency condition that optimal value functions must satisfy. Specifically, the optimal state-value function is defined as the maximum expected return achievable by selecting the best possible action in a given state. Similarly, the optimal action-value function is expressed as the immediate reward plus the highest expected future return obtainable from the subsequent state [18]. The Bellman Optimality Equation serves as the theoretical cornerstone for a wide range of optimal control algorithms used in reinforcement learning.

2.4.2 Types of Reinforcement Learning

Several methods have been developed to solve Markov Decision Processes (MDPs), differing mainly in how they use information about the environment. These approaches can be broadly classified into model-based and model-free methods. Model-based methods rely on a known or learned model of the environment to predict future states and rewards, allowing agents to plan their actions in advance. In contrast, model-free methods do not assume any prior knowledge of the environment and instead learn optimal behaviors directly through experience and interaction. Understanding this distinction is fundamental for analyzing how different reinforcement learning algorithms operate and adapt to uncertainty.

Model-Based Reinforcement Learning

Dynamic Programming Methods Dynamic programming is a model-based approach and solves MDPs by iteratively improving value functions through:

- **Policy Iteration** alternates between policy evaluation and policy improvement until the policy converges to the optimal policy.
  - *Policy evaluation* consists of estimating the value function associated with a given policy π, typically by computing the expected return obtained when the agent follows that policy from each state. This step answers the question of how good the current policy is, by assessing its long-term performance in the environment.
  - *Policy improvement*, on the other hand, aims to enhance the current policy by exploiting the information obtained during policy evaluation. In this phase, the policy is updated by selecting actions that yield higher expected returns according to the evaluated value function, resulting in a new policy that is guaranteed to be at least as good as the previous one [18].
- **Value Iteration:** updates the value function until it converges to the optimal value function.

$$V_{k+1}(s) = \max_{a \in A} \sum_{s' \in S} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_k(s') \right]$$

  where:
  - k is the iteration index of the algorithm;

- $V_k(s)$ is the estimated value of state s at iteration k;
- $V_{k+1}(s)$ is the updated value estimate at the next iteration;
- A is the set of available actions;
- S is the set of possible next states;
- $P(s' \mid s, a)$ is the transition probability to state s' given state s and action a;
- $R(s, a, s')$ is the reward received after transitioning to s';
- $\gamma \in [0, 1)$ is the discount factor.

Dynamic programming breaks down complex tasks into smaller subtasks. It then models problems as workflows of sequential decisions made in discrete time steps. Each decision is made based on the possible resulting next state, since dynamic programming considers potential future states and reward signals when selecting actions. Indeed, dynamic programming is a value-based method, meaning it selects actions based on their estimated values according to a policy that aims to maximize its value function. An agent's reward R for a given action is defined as a function of that action a, the current environmental state s, and the potential next state s':

$$R(s, a) = \sum_{s' \in S} R(s, a, s')\, P(s' \mid s, a)$$

where $P_t(s' \mid s, a)$, called the *transition probability function*, denotes the probability that the system is in state s' at time $t+1$ when the agent chooses action a in state s at time t [9]. Dynamic programming seeks to maximize cumulative rewards by consistently selecting the best possible actions at each state, using recursive relationships such as the Bellman equations [17]. Dynamic programming theory guarantees the existence of at least one optimal stationary policy. However, classical dynamic programming techniques require full knowledge of the reward function and transition probabilities, which are generally unavailable in real-world scenarios.

Model-Free Reinforcement Learning

Monte Carlo methods The Monte Carlo method assumes a black-box environment, making it a model-free approach. Since no prior knowledge of the environment is available, Monte Carlo methods rely entirely on direct interaction to gather information, so these methods are purely experience-driven. These methods sample sequences of states, actions and rewards exclusively through interaction with the environment. As a result, learning

occurs through trial and error [18] rather than through predefined probability distributions or transition models. This experiential learning process enables Monte Carlo methods to adapt to unknown environments, but it also requires a large number of episodes to achieve reliable performance. Monte Carlo methods estimate value functions by computing the average return for each state-action pair based on observed episodes. Consequently, Monte Carlo methods must wait until an entire episode (or planning horizon) is completed before updating their value estimates and policies. Within this framework, two main variants can be distinguished: First-Visit and Every-Visit Monte Carlo.

- **First-Visit Monte Carlo** estimates the value of a state by averaging the returns obtained only on the first visit to that state in each episode. This avoids updating the same state multiple times within the same episode, reducing the risk of bias in the results.
- **Every-Visit Monte Carlo**, on the other hand, considers all visits to a state within each episode. The state's value is estimated by averaging the returns obtained in each visit. This method uses more data than First-Visit but may introduce greater correlation between observations.

In both variants, the value function is updated according to:

$$V(s) \leftarrow V(s) + \alpha \left[G_t - V(s) \right]$$

where $V(s)$ is the estimated value of state s, α is the learning rate, and G_t is the actual return obtained from time step t.

Temporal Difference learning Temporal-Difference (TD) learning is a hybrid approach that combines the concepts of dynamic programming and Monte Carlo methods. It relies on experience to learn from interaction with the environment as Monte Carlo method, indeed is model-free approach. It also incorporates the bootstrapping idea from dynamic programming by updating value estimates incrementally, based on existing predictions and after each transition, without waiting for the final outcome of an episode. In this way, TD learning algorithms adjust the estimated value of a state and consequently the agent's policy, incrementally after each step, enabling faster and more continuous learning compared to waiting for complete episodes. As its name suggests, a TD learning agent adjusts its policy based on the difference between the predicted reward and the reward received at each state. In other words, while both dynamic programming and Monte Carlo methods focus on the rewards obtained, TD learning additionally evaluates the prediction error

between expected and observed rewards. This difference, known as the temporal difference error, is then used to update value estimates for subsequent states immediately, without waiting for the episode to terminate.

$$V(s_t) \leftarrow V(s_T) + \alpha \left[R_{t+1} + \gamma V(s_{t+1}) - V(s_t) \right]$$

where:

- $V(s_t)$ is the estimated value of state s at time step t;
- $\alpha \in (0, 1]$ is the learning rate, controlling the update step size;
- R_{t+1} is the reward received after transitioning from s_t;
- $\gamma \in [0, 1)$ is the discount factor;
- $V(s_{t+1})$ is the estimated value of the next state s_{t+1}.

The temporal difference error δ_t is defined as:

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

so that the update rule can be written compactly as:

$$V(s_t) \leftarrow V(s_t) + \alpha \, \delta_t$$

TD learning exists in multiple variants, among which State-Action-Reward-State-Action (SARSA) and Q-learning are two of the most prominent.
SARSA is an on-policy TD algorithm, meaning it evaluates and improves the same policy that it follows during learning, therefore updates the action-value function based on the action taken by the current policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

where:

- $Q(s_t, a_t)$ is the estimated value of taking action a_t in state s_t;
- $\alpha \in (0, 1]$ is the learning rate;
- R_{t+1} is the reward received after the transition;
- $\gamma \in [0, 1)$ is the discount factor;
- $Q(s_{t+1}, a_{t+1})$ is the estimated value of the next state-action pair, where a_{t+1} is the action chosen by the **current policy**.

Q-Learning is a model-free, off-policy reinforcement learning algorithm that aims to directly approximate the optimal action-value function, independently of the policy used to generate behavior [19]. Q-Learning separates the behavior policy, which directs exploration to gather new experiences, from the target policy, which focuses on exploiting the current value estimates to guide optimal decisions. The behavior policy may include exploratory mechanisms such as ε-greedy action selection, while the target policy is implicitly defined as the greedy policy with respect to the learned action-value function. This separation allows Q-Learning to learn the optimal policy regardless of how the agent behaves during data collection, provided that sufficient exploration occurs. For this reason, the algorithm is often described as exploration-insensitive, meaning that its convergence properties do not depend on the specific exploration strategy adopted, as long as all state-action pairs are visited sufficiently often [9]. The core update rule of Q-Learning is derived from the Bellman optimality equation and is defined as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

where:

- α is the learning rate,
- γ is the discount factor,
- R_{t+1} is the reward received after taking action a_t in state s_t,
- $\max_a Q(s_{t+1}, a)$ represents the maximum estimated future return obtainable from the next state.

The algorithm updates the action-value function using the maximum estimated reward of the subsequent state, allowing it to iteratively improve its policy based on expected future returns. Since Q-Learning does not require explicit knowledge of the transition probabilities or reward function, it can learn directly from raw experience, making it particularly suitable for environments where a model is unavailable or difficult to construct [20]. Once the optimal action-value function $Q^*(s, a)$ has been learned, the optimal policy is obtained by selecting, in each state, the action that maximizes the estimated Q-value:

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

For convergence to the optimal solution, it is necessary that every state-action pair is visited infinitely often and that the learning rate decreases appropriately over time. Under these conditions, also called stochastic approximation

conditions, the action-value estimates are guaranteed to converge to their optimal values [19].

2.4.3 Deep Q-Networks

Deep Q-Networks (DQN), introduced by Mnih et al. in 2015 [17], revolutionized reinforcement learning by demonstrating that deep neural networks could successfully approximate Q-functions in high-dimensional state spaces. Prior to DQN, Q-learning was limited to small, discrete state spaces due to its reliance on tabular representations. By integrating convolutional neural networks (CNNs) with Q-learning [17], DQN enabled end-to-end learning directly from raw sensory inputs, such as pixel-based game screens. This innovation allowed reinforcement learning agents to achieve human-level performance on several Atari 2600 games using only visual input and game scores as feedback, without any handcrafted features. The DQN algorithm addresses two key challenges that arise when combining Q-learning with neural networks: instability due to correlations in the sequence of observations and non-stationarity of the target values used for training. In standard online reinforcement learning, consecutive observations are highly correlated. Training a neural network on sequential samples violates the independent and identically distributed (i.i.d.) assumption required for stable stochastic gradient descent, leading to oscillations or divergence. In addition, Q-learning relies on bootstrapped targets derived from the same network being trained. As network parameters change during learning, the target values also change, resulting in a moving target problem that destabilizes training. DQN employs two critical innovations to stabilize learning [17]. The first innovation was inspired by biological mechanism, the experience replay stores agent transitions, at each time-step, in a replay buffer. In addition, during training, samples minibatches uniformly for training. This technique breaking temporal correlations between past experience, enabling more efficient use of past experience and stabilizes learning through randomized training batches. This allows gradient updates to better approximate the true expectation of the Bellman equation. The second innovation was targeting networks. This technique maintains a separate copy of the Q-network whose parameters are updated less frequently, providing more stable target values for the loss function, this help to reduces harmful feedback loops and prevents divergence. These techniques enable successful learning in complex domains such as Atari 2600 games, where the state space consists of raw pixel inputs.

Loss Function and Optimization The DQN loss function is designed to minimize the difference between the current Q-value estimates and the

target values defined by the Bellman optimality equation. Using a target network with parameters θ^- , the algorithm stabilizes learning by providing a fixed reference for the updates. In practice, DQN can be seen as a method that approximates the Q-function with a deep neural network and iteratively solves the Bellman optimality equation via stochastic gradient descent.

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim \mathrm{U}(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

where:

- $L(\theta)$ is the loss function to be minimized with respect to the network parameters θ;
- $(s, a, r, s') \sim \mathrm{U}(D)$ denotes that the transitions are sampled uniformly from the replay buffer D;
- r is the immediate reward received after taking action a in state s;
- $\gamma \in [0, 1)$ is the discount factor;
- θ_i^- are the parameters of the target network, kept fixed during updates to stabilize training, at iteration i;
- $\max_{a'} Q(s', a'; \theta_i^-)$ is the maximum Q-value over all possible next actions a', estimated by the target network;
- $Q(s, a; \theta_i)$ is the Q-value estimated by the online network for the current state-action pair (s, a);
- $r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i)$ is the TD error, measuring the discrepancy between the current estimate and the bootstrapped target based on the best possible next action [5].

One of the most significant aspects of DQN is its ability to learn directly from raw sensory inputs, requiring little to no prior domain knowledge or handcrafted feature engineering. Experimental results show that the DQN agent is capable of achieving performance levels comparable to, and in some cases exceeding, those of human players across a wide range of Atari 2600 games [17]. These findings highlight the robustness of deep reinforcement learning methods, showing that complex behaviors can emerge solely from interaction with the environment and scalar reward feedback. The model's success across diverse gaming environments indicates its potential applicability to broader decision-making problems beyond the Atari benchmark.

2.5 Atari-like Environments as Testbeds

Over the past decade, two-dimensional video games, especially those from the Atari 2600 game suite, have become standard benchmarks for the evaluation of reinforcement learning and artificial intelligence systems [21]. By offering high-dimensional visual inputs, diverse dynamics, and well-defined evaluation protocols within a controlled setting, these environments bridge the gap between simple toy problems and real-world applications. This section examines the characteristics and inherent challenges that make Atari-like environments particularly suitable as experimental testbeds for AI research.

2.5.1 Characteristics and Challenges of two-dimensional Game Environments

Atari games present diverse challenges that require different capabilities [22]. In this study, we concentrate our analysis on the Atari game Breakout as a representative benchmark environment. This choice allows us to highlight several fundamental properties shared by many Atari games suit. More in details, these environments often require strong temporal reasoning. Temporal reasoning is essential in many games, where success depends on understanding sequences of events, anticipating future states, and planning multi-step strategies. In some cases, precise timing of actions is crucial, while in others, rewards are delayed and the consequences of decisions become apparent only after multiple steps. Breakout requires predicting where the ball will land based on its current trajectory and velocity. Furthermore, the agent must plan the paddle's movements several steps in advance, especially when the ball bounces off bricks or edges. This corresponds to the need for multi-step planning and anticipation of future states. Exploration and credit assignment pose additional challenges. Many Atari games sparse reward structures, in which positive feedback is infrequent, making difficult for learning algorithms to discover effective strategies. Furthermore, long action sequences with delayed outcomes complicate the attribution of credit to individual actions that ultimately lead to success or failure. Partial observability is another important characterizes several games, where the full state is not visible in a single frame. This requires agents to integrate information over time or maintain memory of past observations. In Breakout, even though the agent receives numerical information such as the position and speed of the ball, it does not know the complete future trajectory or any changes in speed due to multiple bounces. This necessitates mechanisms for temporal integration beyond simple reactive policies. Generalization across

games remains a key challenge [22] for the development of general-purpose learning systems. Although deep reinforcement learning has achieved superhuman performance on individual games, transferring of learned knowledge across different games remains difficult. Each game may require different visual features, temporal abstractions, and strategic reasoning, thereby testing the generality and robustness of learning algorithms. Even though Breakout doesn't directly test your ability to transfer strategies to other games, while the goal is to develop a generic algorithm, generalization remains a theoretical challenge of the Atari suite. The diversity and complexity of Atari games, combined with their standardized interface and computational tractability, make them ideal testbeds [21] for developing and evaluating artificial intelligence systems. They provide a middle ground between simple toy problems and real-world applications, offering sufficient complexity to be challenging while remaining amenable to systematic experimentation. The characteristics are particularly relevant to the framework proposed in this thesis, which aims to extract interpretable symbolic knowledge from visual observations and use this knowledge to guide learning. The structured nature of 2D games, characterized by discrete objects obeying consistent rules, provides a suitable domain for testing the hypothesis that combining perceptual processing, symbolic reasoning, and reinforcement learning can lead to more efficient, interpretable, and generalizable AI systems.

3 State of the Art and Challenges

3.1 Challenges in Symbolic Interpretation

3.1.1 The Illusion of Understanding

The Lack of World Models

3.1.2 The Black Box Problem

Making Sense of AI: Toward Explainability

3.1.3 Learning Paradigms and Explainable AI

Learning Without Understanding

Explainable AI: Goals and Approaches

Post-hoc Explainability

Intrinsic Interpretability

Why XAI is Not Enough

3.2 Cognitive Foundations

3.2.1 Human World Modelling and the Origins of Structured Cognition

3.2.2 Core Knowledge Theory: Cognitive Principles and Psychological Foundations

3.2.3 Definition and Relevance for World Understanding

3.2.4 Cognitive Models and Symbolic Representation

3.3 Positioning of the Proposed Framework

4 Framework Design - Proposed Framework

4.1 Conceptual Foundations

4.1.1 Evolution of the Idea

4.2 Proposed Implementation

4.2.1 Segmentation

4.2.2 Symbolic System as Knowledge Construction Module

4.2.3 From Symbolic Rules to a Playable Environment

4.2.4 Generic Environment as a Transitional Representation

4.2.5 Deep Q-Network in a Domain-Agnostic Environment

5 Implementation

5.1 End-to-End Pipeline

5.2 General Architecture of the Framework

5.2.1 Design Choices

5.2.2 Selected Techniques and Algorithms

5.2.3 Modular Structure of the System

Video Acquisition and Preprocessing Module

Feature Extraction Module

Symbolic Mapping Module - Symbolic Reasoning Engine

Integration of Core Knowledge for Semantic Understanding

RL Agent

5.2.4 Optimizations and Efficiency Considerations

6 Experimental Evaluation

6.1 Datasets and Testing Scenarios

6.1.1 Evaluation Metrics: Accuracy, Interpretability, Scalability

6.2 Experimental Results

6.2.1 Comparison with Existing Methods

6.2.2 Critical Discussion of the Results

7 Applications and Future Perspectives – Conclusion

7.1 Future Directions

7.1.1 Potential Applications of the Framework

7.1.2 Possible Extensions of the Work

7.1.3 Open Challenges and Future Research Directions

7.2 Summary of Contributions, Impact of the Research, and Final Reflections

References

Bibliography

[1] B. Liang, Y. Wang, and C. Tong, "Ai reasoning in deep learning era: From symbolic ai to neural–symbolic ai," *Mathematics*, vol. 13, no. 11, p. 1707, 2025.

[2] G. Marcus, "Deep learning: A critical appraisal," *arXiv preprint arXiv:1801.00631*, 2018. [Online]. Available: https://arxiv.org/abs/1801.00631

[3] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Networks*, vol. 61, pp. 85–117, 2015.

[4] A. S. d'Avila Garcez, L. C. Lamb, and D. M. Gabbay, *Neural-Symbolic Cognitive Reasoning*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. [Online]. Available: https://link.springer.com/book/10.1007/978-3-540-73246-4

[5] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015. [Online]. Available: https://doi.org/10.1038/nature14539

[6] A. S. d'Avila Garcez, T. R. Besold, L. De Raedt, P. Földiak, P. Hitzler, T. Icard, and D. L. Silver, "Neural-symbolic learning and reasoning: Contributions and challenges," in *AAAI Spring Symposia*, Mar. 2015, pp. 18–21. [Online]. Available: https://www.aaai.org/Library/Symposia/Spring/ss15-01.php

[7] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman, "Building machines that learn and think like people," *Behavioral and Brain Sciences*, vol. 40, p. e253, 2017.

[8] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.

[9] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 2014.

[10] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[11] H. Bay, T. Tuytelaars, and L. Van Gool, "Surf: Speeded up robust features," in *European Conference on Computer Vision (ECCV)*. Springer, 2006, pp. 404–417.

[12] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893.

[13] C. Harris and M. Stephens, "A combined corner and edge detector," in *Proceedings of the Alvey Vision Conference*, 1988, pp. 147–151.

[14] E. Rosten and T. Drummond, "Machine learning for high-speed corner detection," in *European Conference on Computer Vision (ECCV)*. Berlin, Heidelberg: Springer Berlin Heidelberg, May 2006, pp. 430–443.

[15] G. Van Houdt, C. Mosquera, and G. Nápoles, "A review on the long short-term memory model," *Artificial Intelligence Review*, vol. 53, no. 8, pp. 5929–5955, 2020. [Online]. Available: https://doi.org/10.1007/s10462-020-09825-6

[16] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Englewood Cliffs: Prentice-Hall, 1995.

[17] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2015.

[19] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992. [Online]. Available: https://doi.org/10.1007/BF00992698

[20] L. Huang *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, Jan. 2025. [Online]. Available: http://dx.doi.org/10.1145/3703155

[21] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013. [Online]. Available: https://doi.org/10.1613/jair.3912

[22] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling, "Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents," *Journal of Artificial Intelligence Research*, vol. 61, pp. 523–562, 2018. [Online]. Available: https://doi.org/10.1613/jair.5699

A Additional Technical Details

B Code Snippets or Pseudocode of Relevant Modules

C Dataset Specifications or Experimental Settings